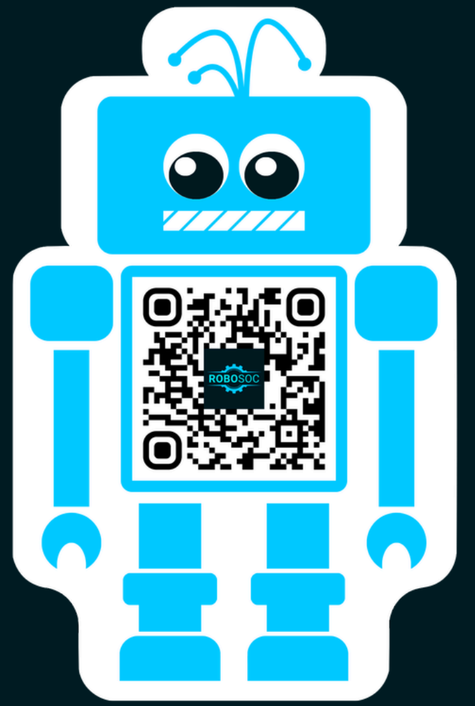
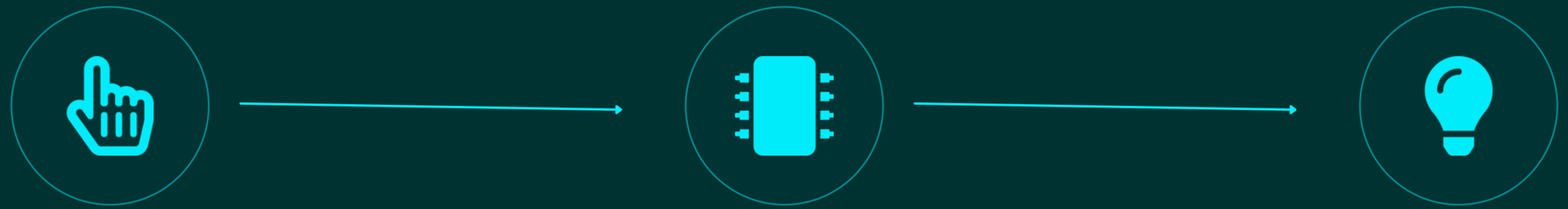

ROBOSOC

ROBOSOC

Workshop 4: Embedded systems



What is an embedded system?



INPUT

buttons, switches, sensors

BRAIN

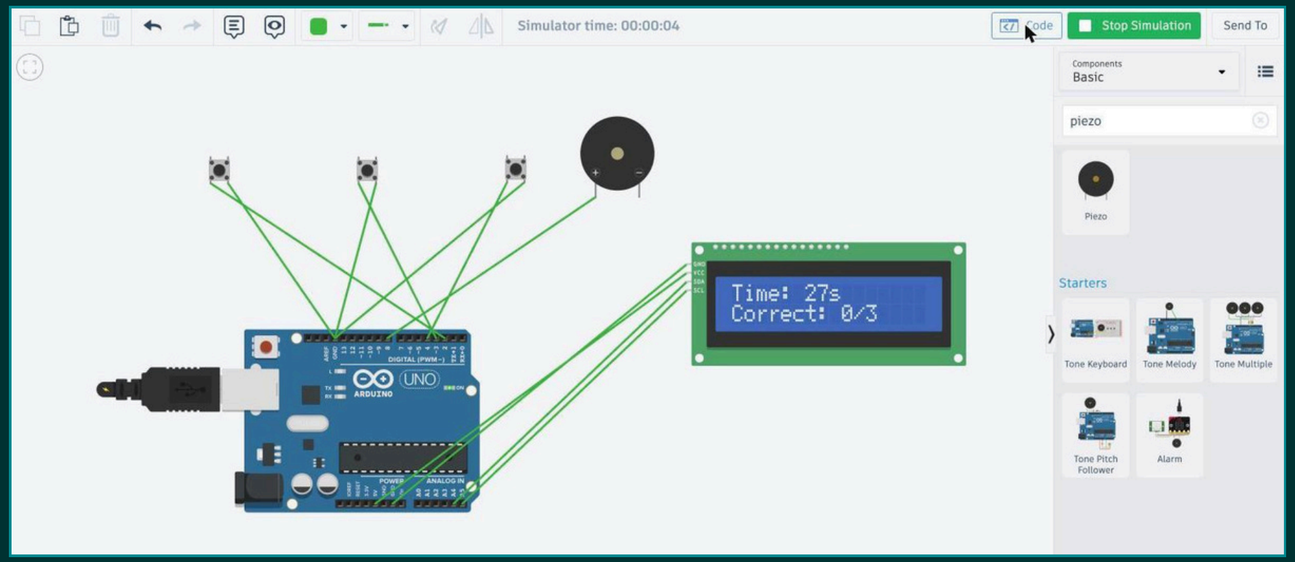
an Arduino running your code

OUTPUT

LEDs, screens, buzzers, motors

- A computer inside a thing, doing one job.
- Washing machine: door switch → program → motor, beep.
- E-scooter: throttle → controller → motor.
- Today: an Arduino Uno, simulated in Tinkercad. The code is real.

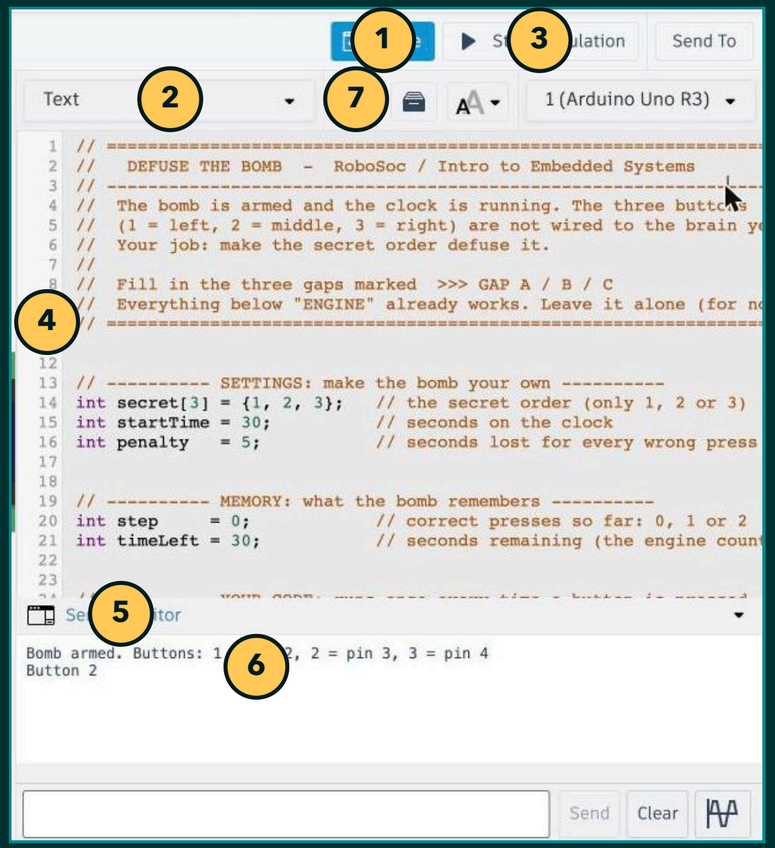
Tinkercad Circuits



- Free circuit simulator in the browser.
- Drag parts, wire them, write code

Code opens the editor. **Start / Stop Simulation** compiles and runs. **Serial Monitor** shows the board's text output.

The code editor



1. Code: open / close the panel.
2. Text, not Blocks.
3. Start / Stop Simulation. Stop before you edit.
4. Line numbers. 29:5 = line 29, column 5.
5. Serial Monitor: click to open.
6. The board's output.
7. Download the .ino for the real Arduino IDE.

The Serial Monitor

```
void setup() {
  Serial.begin(9600);               // open the line at 9600 baud
  Serial.println("Bomb armed.");    // println = print + new line
}

void loop() {
  Serial.print("Time left: ");      // print = stay on the same line
  Serial.println(timeLeft);         // numbers and variables work too
}
```

Serial Monitor

```
Bomb armed.
Time left: 30
Time left: 29
Time left: 28
Button 2
```

- Serial.begin(9600) in setup(). println = new line, print = same line.
- Open it from the bar at the bottom of the code panel.
- Stuck? Print the variable and read what it really is.
-
- Print on events, not every loop, or it floods and slows the simulator.

setup() and loop()

```
void setup() {                  // runs ONCE at power-on
  pinMode(13, OUTPUT);          // pin 13 drives the on-board LED
}

void loop() {                   // runs FOREVER, again and again
  digitalWrite(13, HIGH);       // LED on
  delay(500);                   // wait half a second
  digitalWrite(13, LOW);        // LED off
  delay(500);
}
```

- setup() runs once at power-on.
- loop() runs forever,

Variables

```
int timeLeft = 30;        // a box called timeLeft, holding 30
int penalty  = 5;

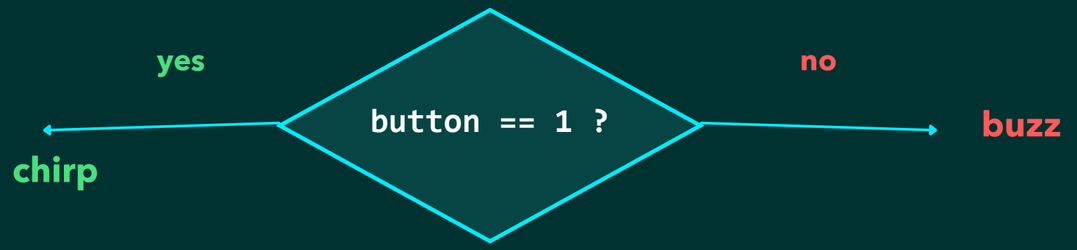
timeLeft = timeLeft - penalty;   // read 30, take 5 away, store 25

int secret[3] = {1, 2, 3};       // a list: three boxes in a row
secret[0]                        // the first box is number 0  -> 1
secret[step]                     // the box that step points at
```

`+` add `-` subtract `*` multiply `/` divide `=` put in the box

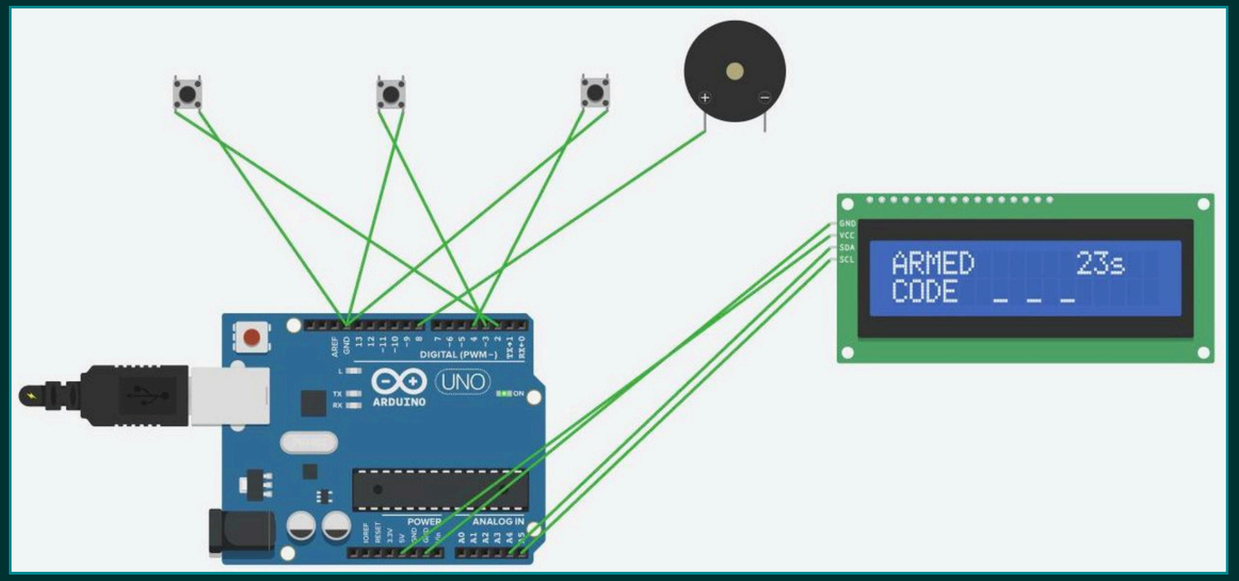
If statements

```
if (button == 1) {          // is button equal to 1?
  beep(1200, 80);           // yes: chirp
} else {
  beep(150, 300);           // no: buzz
}
```

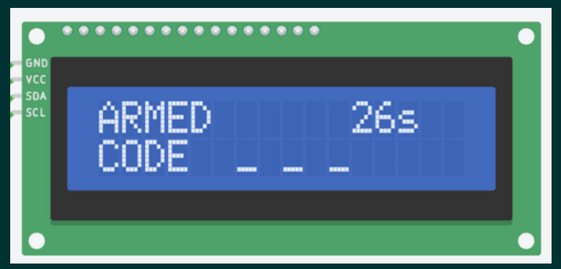


`==` equal `!=` not equal `<` `>` `<=` `>=` `&&` and `||` or `!` not

Game 1: Defuse the bomb



- 30 s on the clock.
- Three buttons, secret order.
- Wrong press: 5 s gone, start again.
- Zero: boom



armed, 26 s to go



two correct, one to go



defused: the time left is your score

Game 1: your task

```
void onButton(int button) {
  if (None) {           // fill in: button same as secret[step]?
    step = None;        // fill in: step up by one
    beep(1200, 80);
    if (step == 3) defused();
  } else {
    timeLeft = None;    // fill in: timeLeft down by penalty
    step = 0;
    beep(150, 300);
  }
}
```

Fill in the three Nones

- if (None) is button the same as secret[step]?
- step = None step goes up by one.
- timeLeft = None timeLeft goes down by penalty.

Then

- Test it.
- Change secret, startTime, penalty.
- Swap laptops and defuse theirs.

Game 1: answer

```
void onButton(int button) {
  if (button == secret[step]) {
    step = step + 1;
    beep(1200, 80);
    if (step == 3) defused();
  } else {
    timeLeft = timeLeft - penalty;
    step = 0;
    beep(150, 300);
  }
}
```

- `if (None)` → `if (button == secret[step])`
- `step = None;` → `step = step + 1;`
- `timeLeft = None;` → `timeLeft = timeLeft - penalty;`

- == compares. A single = is the usual mistake.
- secret[step] moves along the list as step grows.

Game 1: extension

A. Hint after two mistakes

Count wrong presses. On the second one, print the first secret button to the Serial Monitor.

```
int mistakes = 0;

  } else {
    timeLeft = timeLeft - penalty;
    step = 0;
    beep(150, 300);
    // write this: count the mistake.
    // on the 2nd one, Serial.println(secret[0]);
  }
```

B. Random secret every game

Fill the secret list with random buttons when the board starts, so every game is different.

```
void setup() {
  randomSeed(analogRead(A0));
  // write this: a for loop that puts
  // random(1, 4) into secret[0], secret[1], secret[2]
  ...
}
```

Harder two players take turns: the clock resets and the secret changes after each defuse. Keep a score for each player.

The markers are already in your code: search for "extension".

Functions and loops

```
void feed() {                  // define it once
  happiness = happiness + 2;
  say("Yum!");
}

feed();                        // call it: runs the block
feed();                        // call it again: +2 more
```

```
for (int i = 0; i < 3; i++) {   // i = 0, 1, 2
  beep(1000, 80);
  delay(120);
}
```

i=0 ✓ beep i=1 ✓ beep i=2 ✓ beep i=3 ✗ stop

- can define once and call multiple more times

Game 2: Hack the robot



ANGRY: three pokes since the last meal



HAPPY: fed, happiness 4 or more



SLEEPY: ignored for 10 seconds

- Same circuit, new code.
- 1 feed, 2 poke, 3 play.
- Remembers happiness (0 to 5) and pokes since the last meal.
- Mood rules, checked every second. First true rule wins.

```
if (pokes >= 3)                  return "ANGRY";
if (secondsSinceLastButton() >= 10) return "SLEEPY";
if (happiness >= 4)              return "HAPPY";
if (happiness <= 1)              return "SAD";
return "BORED";
```

Game 2: your task

```
void feed() {
  happiness = None;                    // fill in: up by 2
  ...
void poke() {
  pokes = None;                        // fill in: up by 1
  ...
void play() {
  for (int i = 0; i < None; i++) {     // fill in: 3 times
  ...
String mood() {
  if (None) return "ANGRY";            // fill in: pokes 3 or more
```

Check 3 pokes = ANGRY. Feed = HAPPY. 10 s alone = SLEEPY.

Done? Extension slide: a fourth mood, or hunger.

Fill in the four Nones

- feed: happiness goes up by 2.
- poke: pokes goes up by 1.
- play: the loop runs 3 times.
- mood: ANGRY when pokes is 3 or more.

Hack it

Add one secret behaviour. Feed it five times → SICK. Play while ANGRY → calm. Poke it asleep → FURIOUS. Swap: 60 s to find it.

Game 2: answer

```
void feed() {
  happiness = happiness + 2;
  ...
void poke() {
  pokes = pokes + 1;
  ...
void play() {
  for (int i = 0; i < 3; i++) {
  ...
String mood() {
  if (pokes >= 3) return "ANGRY";
```

- happiness = None; → happiness = happiness + 2;
- pokes = None; → pokes = pokes + 1;
- i < None → i < 3
- if (None) → if (pokes >= 3)

- The engine clamps happiness to 0 to 5.
- >= 3: a fourth poke keeps it angry. == 3 would not.

Game 2: extension

A. A fourth mood

Five feeds in a row makes it SICK. Poking or playing resets the count.

```
int feeds = 0;

void feed() {
  // write this: feeds goes up by 1
  ...
String mood() {
  // write this: the first rule. 5 feeds or more → "SICK"
  ...
String faceFor(String m) {
  // write this: a face for SICK
```

B. Hunger

Every 10 seconds without food, happiness drops by 1.

```
unsigned long lastFedAt = 0;

void feed() {
  // write this: remember millis() in lastFedAt
  ...
void loop() {
  // write this: if 10000 ms have passed since lastFedAt,
  // happiness goes down by 1 and lastFedAt restarts
  ...
```

Harder a different tune for every mood: a short for loop of beeps in draw() when the mood changes.

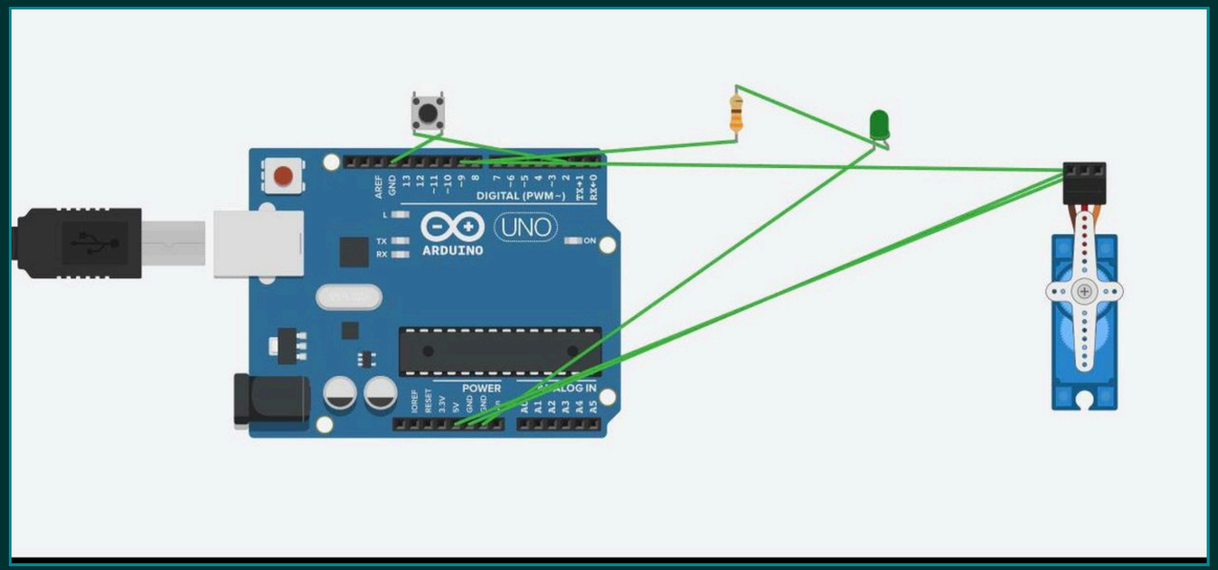
The markers are already in your code: search for "extension".

Timing with millis()

```
unsigned long now = millis();           // e.g. 12000 ms since power-on
if (now - lastKnockAt >= 1500) {        // 12000 - 9800 = 2200 ms of silence
  onPause();                            // the attempt is over: check the count
}
```

- delay(1500) freezes everything for 1.5 s. Nothing is read, nothing updates.
- millis() = milliseconds since power-on. Never stops.
- Remember when something happened, then keep asking how long ago. loop() keeps running.
- The lock needs this: 1.5 s of silence after the last knock means the attempt is over.

Game 3: Secret knock lock



- Knock = press the button.
- 3 knocks, then 1.5 s of silence → servo to 90°, LED on, 3 s, then locked again.
- Wrong count → LED flashes once.

- **Button** pin 2 and GND.
- **LED** pin 8 → 330 Ω → long leg. Short leg → GND.
- **Servo** brown GND, red 5V, orange pin 9.

- Build it and code it from scratch: Create → Circuits, Arduino Uno R3 plus the three parts.
- Hover a leg for its name. Click a leg, then a pin, to wire. R rotates. Click the resistor to set 330 Ω.

Game 3: your task

- Build the circuit.
- Write the code: count presses, wait for 1.5 s of silence, compare with the secret, open the door.
- Test: 2 knocks → flash. 3 → open. 4 → flash.

Stuck? The Game 3 helper link has the code with three Nones. You still build the circuit.

New bits

```
#include <Servo.h>
Servo door;
door.attach(9);
door.write(90);                  // 0 = locked, 90 = open

pinMode(2, INPUT_PULLUP);        // pressed = LOW
bool down = (digitalRead(2) == LOW);
if (down && !wasDown) { ... }    // a new press
wasDown = down;
```

Game 3: answer

```
#include <Servo.h>
Servo door;
int secretKnocks = 3, knocks = 0;
unsigned long lastKnock = 0;
bool wasDown = false;
void setup() {
  pinMode(2, INPUT_PULLUP);
  pinMode(8, OUTPUT);
  door.attach(9);
  door.write(0);
}
void loop() {
  bool down = (digitalRead(2) == LOW);
  if (down && !wasDown) {
    knocks = knocks + 1;
    lastKnock = millis();
  }
  wasDown = down;
  if (knocks > 0 && millis() - lastKnock > 1500) {
    if (knocks == secretKnocks) {
      door.write(90); digitalWrite(8, HIGH);
      delay(3000);
      door.write(0);  digitalWrite(8, LOW);
    } else {
      digitalWrite(8, HIGH); delay(150); digitalWrite(8, LOW);
    }
    knocks = 0;
  }
}
```

- Press detector: down now and not down last time.
- Each press restarts the 1.5 s silence timer.
- After the silence: right count opens for 3 s, wrong count flashes. Either way knocks goes back to 0.
- Anything that passes the three tests is a correct answer.

Game 3: extension

A. Locked light

Wire a red LED: pin 7 → 330 Ω → long leg, short leg → GND. On while locked, off while the door is open.

```
void setup() {
  ...
  // extension A: pinMode(7, OUTPUT); turn it on: locked
}
    if (knocks == secretKnocks) {
      door.write(90); digitalWrite(8, HIGH);
      // extension A: red LED off while open, on again
after
      delay(3000);
```

B. Sound

Wire a piezo: + to pin 6, – to GND. A high note when it opens, a low buzz when the count is wrong.

```
    if (knocks == secretKnocks) {
      // extension B: tone(6, 880, 200) when it opens
      ...
    } else {
      // extension B: tone(6, 150, 400) when wrong
    }
```

REMINDER!

YOU MUST PURCHASE A MEMBERSHIP IN ORDER TO ATTEND WORKSHOPS AND TAKE PART IN COMPETITIONS!!!

YOU WILL ALSO GET PRIORITY FOR SOCIAL EVENTS AND BADMINTON / VOLLEYBALL TEAM

ONLY £5 ON OUR SU PAGE

Join ROBOSOC Today!

ROBOSOC

Competition Team
Sign-Up

Discord

LinkTree
(ALL our links)

Instagram

IMS Team
Sign-Up